

Backend Reliability Challenge: Handle It Smart

Hands-On Learning Report — Module 2: Reliable LLM Middleware

1. Overview

A chat-assistant prototype worked locally but hit HTTP 429 rate limits, timed out randomly, exposed raw provider errors to the frontend, and gave no visibility into token usage. I built a lightweight **FastAPI middleware** exposing a single **POST /api/chat** endpoint that wraps the OpenAI chat-completion call with timeouts, retries with exponential backoff, an in-memory TTL cache, token and retry logging, and a stable JSON response.

Stack: Python, FastAPI, Uvicorn, OpenAI SDK (provider), curl (HTTP client). **Project folder:** `llm_middleware_lab/` (`main.py`, `requirements.txt`, `README.md`). The API key is read from the `OPENAI_API_KEY` environment variable and is never hard-coded. Model and temperature are fixed (`gpt-4o-mini`, `0.2`).

How it was tested: no live OpenAI key was used. The service ran in its built-in **mock provider mode**, which simulates latency, token counts, and injectable failures (429, timeout, 400, 503). This follows the lab tip to simulate errors in code to confirm retry logic. The retry, cache, logging, and error-handling code is the same code path used with the real OpenAI client; only the provider function differs. With `OPENAI_API_KEY` set, the service switches to the real SDK call (not exercised in this report).

2. Design Summary

Requirement	Implementation
Timeout	Per-attempt timeout of 20 s on the OpenAI client (<code>LLM_TIMEOUT_S</code>). SDK built-in retries are disabled (<code>max_retries=0</code>) so the middleware controls all retry behavior.
Retries + exponential backoff	Up to 3 retries after the first attempt. Wait 0.5 s, 1 s, 2 s ($\text{base} \times 2^n$) plus up to 0.1 s random jitter.
What is retried	Only transient failures: HTTP 429, HTTP 5xx, timeouts, connection errors. Never 400/401/403/404/422, because retrying cannot fix them.
Cache	In-memory dict keyed by SHA-256 of model + system prompt + normalized (lower-cased, whitespace-collapsed) user message. TTL 300 s; expired entries are evicted on read.
Logging	Human-readable line per request: timestamp, endpoint, retries, token totals (prompt/completion), latency. Retry attempts and failures are logged separately with the raw provider detail (log only).
Stable response	Same envelope for success and failure: <code>{answer, error, meta}</code> . Errors return a friendly message with a code; raw provider errors never reach the client.
Metrics	<code>GET /api/metrics</code> returns counters: requests, cache hits, retries, failures, token totals, cache hit rate.

3. Test Results (Activity 2)

All requests were sent to `http://localhost:8000/api/chat` while the server logged to the console.

#	Test	Observed result	Verdict
T1	Single request	HTTP 200, answer returned, meta shows 0 retries, 34 prompt + 30 completion tokens, ~400 ms.	Structure matches design
T2	Same request again	HTTP 200, meta.cached=true, 0 tokens, ~1.5 ms round trip (vs 404 ms uncached). Log shows CACHE HIT (saved 64 tokens).	Cache works
T3	Same text, different case and spacing	Cache hit again thanks to key normalization.	Normalization works
T4	Simulated 429 twice, then success	Retries after 0.51 s and 1.06 s; third attempt succeeded. HTTP 200, retry_count=2, latency 2.77 s.	Backoff works
T5	Simulated timeout once	One retry after 0.56 s, then success. retry_count=1.	Timeout retried
T6	Simulated 400 invalid input	No retries (retry_count=0), HTTP 400, friendly message, ~400 ms.	Non-transient errors not retried
T7	Provider down (503 always)	3 retries (0.56, 1.09, 2.09 s), then HTTP 503 with a friendly message after 5.3 s. No raw provider text exposed.	Graceful failure
T8	Malformed payload (wrong field)	HTTP 422 with the same envelope shape and a helpful hint.	Clean validation error

Metrics after the run (GET `/api/metrics`): 7 requests counted (the malformed payload is rejected before reaching the handler), 2 cache hits (hit rate 0.29), 6 retries, 2 failures, 97 prompt tokens and 90 completion tokens consumed. The two cache hits avoided 128 tokens of provider usage (2 x 64) and two provider calls.

4. Output Evidence

Screenshot 1: request and stable JSON response showing the meta block after two retries (terminal capture of the curl call).

```
client request/response - stable JSON with meta block

$ curl -X POST localhost:8000/api/chat -d '{"user_message":"Plan change please [sim-429]}"'
HTTP 200    time=2.78s
{
  "answer": "Yes. You can change your subscription plan at any time; the change
            takes effect at the start of your next billing cycle.",
  "error": null,
  "meta": {
    "retried": true,
    "retry_count": 2,
    "prompt_tokens": 30,
    "completion_tokens": 30,
    "cached": false,
    "latency_ms": 2773
  }
}
```

Screenshot 2: server console logs showing cache hits, backoff retries, token usage, the non-retried 400, and the exhausted-retries failure (terminal capture).

```
server console - token / retry / cache logging

$ uvicorn main:app --port 8000 (MOCK provider, MAX_RETRIES=3, base delay 0.5s, cache TTL 300s)
[2026-10-02 04:59:20] /api/chat - retries=0 tokens=64 (prompt=34, completion=30) latency=400ms
[2026-10-02 04:59:20] /api/chat - CACHE HIT retries=0 tokens=0 (saved 64) latency=0ms
[2026-10-02 04:59:20] /api/chat - CACHE HIT retries=0 tokens=0 (saved 64) latency=0ms
[2026-10-02 04:59:21] /api/chat - transient error (rate_limit status=429); retry 1/3 in 0.51s
[2026-10-02 04:59:22] /api/chat - transient error (rate_limit status=429); retry 2/3 in 1.06s
[2026-10-02 04:59:23] /api/chat - retries=2 tokens=60 (prompt=30, completion=30) latency=2773ms
[2026-10-02 04:59:24] /api/chat - transient error (timeout status=None); retry 1/3 in 0.56s
[2026-10-02 04:59:25] /api/chat - retries=1 tokens=63 (prompt=33, completion=30) latency=1357ms
[2026-10-02 04:59:25] /api/chat - FAILED kind=bad_request status=400 retries=0 latency=400ms detail=invalid input
[2026-10-02 04:59:26] /api/chat - transient error (server_error status=503); retry 1/3 in 0.56s
[2026-10-02 04:59:26] /api/chat - transient error (server_error status=503); retry 2/3 in 1.09s
[2026-10-02 04:59:28] /api/chat - transient error (server_error status=503); retry 3/3 in 2.09s
[2026-10-02 04:59:30] /api/chat - FAILED kind=server_error status=503 retries=3 latency=5341ms detail=service unavailable
```

5. Optimization and Trade-offs

Why this retry strategy and these parameters

Exponential backoff spaces retries out (0.5 s, 1 s, 2 s) so a rate-limited or struggling provider gets breathing room instead of being hammered by immediate retries, which would worsen 429s. **Small random jitter** prevents many clients from retrying at the same instant (a thundering herd). **Three retries** cover most short blips; the worst-case added delay is about 3.5 s of waiting, which is acceptable for a chat UI, whereas more retries would leave users staring at a spinner (T7 already takes 5.3 s). **Only transient errors are retried** (429, 5xx, timeouts, connection errors); 4xx input errors would fail identically each time and waste quota and latency. A **20 s per-attempt timeout** bounds slow model responses, though with 3 retries the worst case is long; a total deadline is a sensible next step.

How caching reduces cost and rate-limit pressure

Every cache hit skips the provider entirely: no tokens are billed, no request counts against the requests-per-minute limit, and the response returns in about a millisecond instead of hundreds of milliseconds to seconds. In the test run, 2 of 7 requests (29 percent) were served from cache. For FAQ-style support traffic, where many users ask near-identical questions, the hit rate would be higher, directly lowering the cost and making traffic spikes less likely to trigger 429s. Normalizing the key (case and whitespace) increases hits without changing meaning.

Trade-offs considered

- **Stale answers:** a cached answer can be outdated if policies change. A 5-minute TTL limits staleness; the TTL should be shorter for volatile content and longer for stable FAQs, and the cache should be cleared when source content changes.
- **Personalization and privacy:** the cache key ignores the user, so it is only safe for non-personalized answers. A real deployment should add user/tenant or conversation context to the key, or skip caching for personal data.
- **Temperature and variety:** caching returns the identical answer each time, which is fine at a low temperature but removes response variety.
- **In-memory limits:** the cache is per process, is lost on restart, is not shared across multiple workers, and grows without bound. Production would use Redis (shared, persistent, with eviction) or an LRU with a size cap.
- **Complexity vs. benefit:** retries and caching add code, state, and edge cases (for example, retrying non-idempotent calls, cache invalidation). The wrapper is kept small and focused to stay maintainable.
- **Latency on failure:** backoff improves success rate but increases worst-case latency; a circuit breaker or a total request deadline would fail fast during long outages.
- **Concurrency:** the endpoint is synchronous and uses `time.sleep` during backoff, which occupies a worker thread. An async implementation (`AsyncOpenAI` and `asyncio.sleep`) would scale better under heavy load.
- **Duplicate in-flight requests:** two identical requests arriving simultaneously both miss the cache. Request coalescing could fix this.

Next steps

Move the cache to Redis, add a total-deadline and circuit breaker, expose Prometheus-style metrics, add per-user rate limiting at the middleware, and run the same tests against the live OpenAI API with a real key.

6. Source Code: main.py

```
"""
LLM Reliability Middleware (FastAPI)
-----
POST /api/chat -> wraps an OpenAI chat-completion call with:
* request timeout
* retries with exponential backoff (429 / 5xx / timeouts / connection errors only)
* in-memory cache with TTL
* token + retry logging
* a stable, frontend-friendly JSON response (never leaks raw provider errors)

Run:  OPENAI_API_KEY=sk-... uvicorn main:app --reload
Test: no key set -> MOCK mode (simulated provider with injectable failures)
"""
import hashlib
import logging
import os
import random
import threading
import time
from typing import Optional

from fastapi import FastAPI, Request
from fastapi.exceptions import RequestValidationError
from fastapi.responses import JSONResponse
from pydantic import BaseModel

# ----- config
API_KEY = os.getenv("OPENAI_API_KEY") # never hard-coded
MODEL = os.getenv("LLM_MODEL", "gpt-4o-mini") # fixed model ...
TEMPERATURE = float(os.getenv("LLM_TEMPERATURE", "0.2")) # ... and temperature
TIMEOUT_S = float(os.getenv("LLM_TIMEOUT_S", "20")) # per-attempt timeout
MAX_RETRIES = int(os.getenv("LLM_MAX_RETRIES", "3")) # retries AFTER first try
BASE_DELAY_S = float(os.getenv("LLM_BASE_DELAY_S", "0.5")) # 0.5s, 1s, 2s
CACHE_TTL_S = int(os.getenv("CACHE_TTL_S", "300")) # 5 minutes
MOCK = API_KEY is None or os.getenv("MOCK_LLM") == "1"

SYSTEM_PROMPT = ("You are a helpful support assistant. Answer briefly and "
                 "accurately. If you are unsure, say so.")

logging.basicConfig(level=logging.INFO,
                    format="[%asctimes] %(message)s", datefmt="%Y-%m-%d %H:%M:%S")
log = logging.getLogger("llm-middleware")

# ----- errors
class ProviderError(Exception):
    """Normalized provider failure so retry logic is provider-agnostic."""
    def __init__(self, status: Optional[int], kind: str, msg: str = ""):
        super().__init__(msg or kind)
        self.status, self.kind = status, kind

def is_retryable(err: ProviderError) -> bool:
    # Transient: rate limit, server errors, timeouts, network problems.
    # NOT retryable: 400/401/403/404/422 -> retrying cannot succeed and wastes quota.
    if err.kind in ("timeout", "connection"):
        return True
    return err.status == 429 or (err.status is not None and err.status >= 500)

# ----- providers
_mock_attempts = {} # message -> attempts seen (to simulate "fail N times then succeed")

def _mock_provider(system: str, user: str):
    """Simulated provider. Trigger failures with tags in the message."""
    n = _mock_attempts.get(user, 0) + 1
    _mock_attempts[user] = n
    time.sleep(0.4) # pretend network/model latency
    if "[sim-400]" in user:
        raise ProviderError(400, "bad_request", "invalid input")
```

```

if "[sim-429]" in user and n <= 2:
    raise ProviderError(429, "rate_limit", "Rate limit reached")
if "[sim-timeout]" in user and n <= 1:
    raise ProviderError(None, "timeout", "request timed out")
if "[sim-down]" in user:
    raise ProviderError(503, "server_error", "service unavailable")
_mock_attempts.pop(user, None)
ptok = max(1, (len(system) + len(user)) // 4)
answer = ("Yes. You can change your subscription plan at any time; the change "
         "takes effect at the start of your next billing cycle.")
return answer, ptok, len(answer) // 4

def _openai_provider(system: str, user: str):
    import openai
    client = openai.OpenAI(api_key=API_KEY, timeout=TIMEOUT_S, max_retries=0) # we own retries
    try:
        r = client.chat.completions.create(
            model=MODEL, temperature=TEMPERATURE,
            messages=[{"role": "system", "content": system},
                     {"role": "user", "content": user}])
    except openai.APITimeoutError as e:
        raise ProviderError(None, "timeout", str(e))
    except openai.APIConnectionError as e:
        raise ProviderError(None, "connection", str(e))
    except openai.APIStatusError as e:
        raise ProviderError(e.status_code, "http_error", str(e))
    return r.choices[0].message.content, r.usage.prompt_tokens, r.usage.completion_tokens

call_provider = _mock_provider if MOCK else _openai_provider

# ----- cache (TTL)
_cache, _cache_lock = {}, threading.Lock()
stats = {"requests": 0, "cache_hits": 0, "retries": 0, "failures": 0,
         "prompt_tokens": 0, "completion_tokens": 0}

def cache_key(system: str, user: str) -> str:
    # normalise whitespace/case so trivially different repeats still hit the cache
    norm = " ".join(user.lower().split())
    return hashlib.sha256(f"{MODEL}|{system}|{norm}".encode()).hexdigest()

def cache_get(key):
    with _cache_lock:
        item = _cache.get(key)
        if item and item["exp"] > time.time():
            return item["val"]
        _cache.pop(key, None) # expired -> evict (avoids stale answers)
    return None

def cache_set(key, val):
    with _cache_lock:
        _cache[key] = {"val": val, "exp": time.time() + CACHE_TTL_S}

# ----- core wrapper
def call_llm_with_retry(system: str, user: str):
    """Returns (answer, meta). Raises ProviderError only after retries are exhausted
    or on a non-retryable error."""
    retries = 0
    while True:
        try:
            answer, ptok, ctok = call_provider(system, user)
            return answer, {"retried": retries > 0, "retry_count": retries,
                           "prompt_tokens": ptok, "completion_tokens": ctok}
        except ProviderError as err:
            if not is_retryable(err) or retries >= MAX_RETRIES:
                err.retries = retries
                raise
            delay = BASE_DELAY_S * (2 ** retries) + random.uniform(0, 0.1) # 0.5, 1, 2 (+jitter)
            retries += 1
            stats["retries"] += 1
            log.warning("/api/chat - transient error (%s status=%s); retry %d/%d in %.2fs",
                       err.kind, err.status, retries, MAX_RETRIES, delay)
            time.sleep(delay)

# ----- API
app = FastAPI(title="LLM Reliability Middleware")

class ChatRequest(BaseModel):

```

```

user_message: str

def envelope(answer, meta, error=None):
    """One response shape for success AND failure -> frontend never branches on format."""
    return {"answer": answer, "error": error, "meta": meta}

@app.exception_handler(RequestValidationError)
async def validation_handler(request: Request, exc: RequestValidationError):
    return JSONResponse(status_code=422, content=envelope(
        None, {"retried": False, "retry_count": 0, "cached": False},
        {"code": "INVALID_REQUEST", "message": "Please send JSON like {\\"user_message\\": \\"...\\\".}"))

@app.post("/api/chat")
def chat(req: ChatRequest):
    t0 = time.time()
    stats["requests"] += 1
    msg = req.user_message.strip()
    if not msg:
        return JSONResponse(status_code=400, content=envelope(
            None, {"retried": False, "retry_count": 0, "cached": False},
            {"code": "EMPTY_MESSAGE", "message": "Please type a message."}))

    key = cache_key(SYSTEM_PROMPT, msg)
    hit = cache_get(key)
    if hit:
        stats["cache_hits"] += 1
        ms = int((time.time() - t0) * 1000)
        log.info("/api/chat - CACHE HIT retries=0 tokens=0 (saved %d) latency=%dms",
            hit["total"], ms)
        return envelope(hit["answer"], {"retried": False, "retry_count": 0, "cached": True,
            "prompt_tokens": 0, "completion_tokens": 0, "latency_ms": ms})

    try:
        answer, meta = call_llm_with_retry(SYSTEM_PROMPT, msg)
    except ProviderError as err:
        stats["failures"] += 1
        ms = int((time.time() - t0) * 1000)
        retries = getattr(err, "retries", 0)
        # Raw provider detail goes to the LOG only, never to the client.
        log.error("/api/chat - FAILED kind=%s status=%s retries=%d latency=%dms detail=%s",
            err.kind, err.status, retries, ms, err)
        retryable = is_retryable(err)
        return JSONResponse(status_code=503 if retryable else 400, content=envelope(
            None, {"retried": retries > 0, "retry_count": retries, "cached": False, "latency_ms": ms},
            {"code": "LLM_UNAVAILABLE" if retryable else "REQUEST_REJECTED",
             "message": ("Our assistant is busy right now. Please try again in a moment."
                if retryable else "We couldn't process that message. Please rephrase it.")}))

    total = meta["prompt_tokens"] + meta["completion_tokens"]
    stats["prompt_tokens"] += meta["prompt_tokens"]
    stats["completion_tokens"] += meta["completion_tokens"]
    cache_set(key, {"answer": answer, "total": total})
    ms = int((time.time() - t0) * 1000)
    meta.update({"cached": False, "latency_ms": ms})
    log.info("/api/chat - retries=%d tokens=%d (prompt=%d, completion=%d) latency=%dms",
        meta["retry_count"], total, meta["prompt_tokens"], meta["completion_tokens"], ms)
    return envelope(answer, meta)

@app.get("/api/metrics")
def metrics():
    """Simple counters for cost/reliability visibility."""
    s = dict(stats)
    s["cache_hit_rate"] = round(s["cache_hits"] / s["requests"], 2) if s["requests"] else 0
    s["mode"] = "mock" if MOCK else "openai"
    return s

```